

Use-Modify-Create Turned “υμοδ ερισδη” by AI: Towards Higher-Level Competences via the Scientific Method

Claus Brabrand

Center for Computing Education Research (CCER)
IT University of Copenhagen
Copenhagen, Denmark

Sue Sentence

Raspberry Pi Computing Education Research Centre
University of Cambridge
Cambridge, UK

1 EXTENDED ABSTRACT & INTRODUCTION

In 2011, Lee et al. proposed the **USE-MODIFY-CREATE (UMC)** framework [3] to describe *staged progression* of student competence. Since then, the UMC framework has become a widely adopted tool for *scaffolding progression* in computing education, especially for education in programming [3]. It consists of three staged learning phases, gradually transitioning from *imitation* (working with artifacts made by others) to *originality* (creating one’s own artifact):

- **USE**: Apply an existing artifact or knowledge as-is:
 - run provided code, follow an example, execute an algorithm.
- **MODIFY**: Adapt or extend artifact to do something different:
 - change parameters, change functionality, add a feature.
- **CREATE**: Design & implement novel artifact from scratch:
 - write original code, make new project, invent unique solution.

The framework has been particularly successful in K–12 computing where it provides an incremental scaffold for students to progress from simple to more complex programming activities [4].

Problem & Opportunity. The rise of generative AI, however, has caused a massive, disruptive shift in competences, impacting UMC. In this paper, we examine that shift, quantifying the *cognitive processes* “lost to automation” in terms of BLOOM’s Taxonomy. If we do nothing, students may simply “outsource” all higher-level activities to AI and, as a result, use *lower-level* cognitive processes. Taking inspiration from a novel *extension* of the UMC framework and **The Scientific Method** itself, we plot a course forward for AI-empowered education that takes us, not *down*, but *up* the competence ladder towards, not *lower-*, but *higher-level* competences.

2 UMC TURNED UPSIDE DOWN BY AI

Since the breakthrough “attention paper” [6], generative AI has been turning education “Upside Down.” This impact is particularly disruptive for UMC. Let us consider how **U-M-C**—in reverse order (as in **C-M-U**)—is impacted by the emergence of generative AI:

Not CREATE! Writing code is no longer the bottleneck; *time-consuming low-level code generation* can be “outsourced” to GenAI.

Maybe MODIFY? Classically, a *human*, *H*, would modify the code directly: $H(\text{code}) = \text{code}'$. With an *AI*, *A*, however, a clever student will quickly realize that the same effect could be achieved by modifying the (input) *prompt* rather than the (output) *code*. Assuming the original prompt gave rise to: $A(\text{prompt}) = \text{code}$, the human may modify the (input) *prompt* as in: $H(\text{prompt}) = \text{prompt}'$, thereby *indirectly* modifying the (output) code: $A(H(\text{prompt})) = \text{code}'$. (A complication is that as you are trying to fix problems in *code*, *non-determinism* may introduce entirely new problems in *code'*. Of course, this could potentially be addressed by supplying both the *code* as well as the intended *modification (delta)* as input to the AI: $A(\text{code}, \text{delta}) = \text{code}'$, akin to CoPilot editing a code-base.)

Definitely USE! Unquestionably, **USE**-competence will be key in the Age of AI; humans (students & professionals) will *use* generative AI. How we go about this—in education—will matter a great deal (as we shall see), but let us first take look at what the changes entail in terms of *cognitive processes* (aka, *competence devaluation*).

3 TAXONOMY OF COGNITIVE PROCESSES

Bloom’s Taxonomy [2], originally proposed in 1956 by Benjamin Bloom and colleagues, was designed as a framework for classifying educational objectives, providing a hierarchical model of cognitive processes to guide curriculum design, assessment, and learning activities. The model distinguishes *six* cognitive processes:

- (1) **REMEMBER** (aka, **KNOWLEDGE**): recognize/recall relevant knowledge from memory;
- (2) **UNDERSTAND** (aka, **COMPREHENSION**): meaning via interpret, summarize, or explain;
- (3) **APPLY** (aka, **APPLICATION**): use knowledge/procedure in new but familiar case;
- (4) **ANALYZE** (aka, **ANALYSIS**): decompose material into individual constituents & explore relationships, incl. structure;
- (5) **SYNTHESIZE** (aka, **CREATE**): put elements together to form a novel, coherent whole often constituting a new product;
- (6) **EVALUATE** (aka, **EVALUATE**): make judgments based on criteria or standards.

Originally, the three highest levels, { **ANALYZE**, **SYNTHESIZE**, **EVALUATE** }, were all considered independent, parallel processes; i.e., all at the top, side-by-side. Later, the processes were reorganized into a *strict hierarchy*, corresponding to the above enumeration (e.g., **SYNTHESIZE** < **EVALUATE**). In 2001, the *Revised BLOOM Taxonomy* [1] proposed a renaming and reorganization.¹

From a BLOOM perspective, UMC is organized as follows:

USE ∈ **APPLY**: learners *apply* given code as-is, *using* it in the context provided by the teacher, often via *running* it; this matches **APPLICATION**, where knowledge is *applied* to concrete tasks (without substantial conceptual transformation).

MODIFY ∈ **ANALYZE** ∪ **SYNTHESIZE**: when learners *modify* code, they must **ANALYZE** its structure and relationships in order to meaningfully *change* it, hereby engaging in (limited) **SYNTHESIZE** by (re-)assembling altered components into a *modified*, functional whole; thus, **MODIFY** incorporates both **ANALYZE** & **SYNTHESIZE**.

CREATE ∈ **SYNTHESIZE**: at this stage learners *create* a novel program (via *design & implementation*), assembling knowledge into something original; this aligns with **SYNTHESIS** (aka, **CREATE**).

¹Renaming from *noun* to *verb* form (e.g., **APPLICATION** to **APPLY**) as well as promoting **SYNTHESIS** (now renamed **CREATE**) to the apex, swapping places with **EVALUATE**.

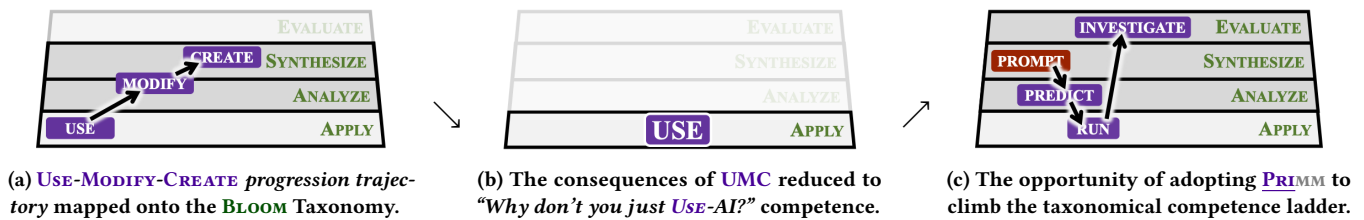


Figure 1: **BLOOM** Taxonomy (upper cognitive processes shown): (a) **UMC** before AI; (b) **UMC** with AI (if we do nothing); and (c) **UMC** with AI & inquiry-based learning.

4 A WAY FORWARD, IN FACT, UP...

Because of generative AI, **UMC** has been undermined; sticking with it would relegate students into predominantly lower-level **USE**-like cognitive processes as in “Why don’t you just **USE** AI?”—a far too common phrase today—“outsourcing” all higher-level thinking to the machine. This scenario is depicted in Figure 1b with students effectively moving down the competence hierarchy towards lower-level cognitive processes; in fact, mainly, **USE** (of AI). However, by “outsourcing” time-consuming lower-level code generation to AI, the human (student and professional) programmer is free to work at a higher-level of abstraction. Herein lies an opportunity to turn “unreflected **USE**” into “reflective **USE**” (of generative AI).

PRIMM. In 2019, Sentence et al. proposed **PRIMM** which could be seen as a refinement of **UMC**, adding more structure to the initial stages (elaborating **USE**) and more emphasis on code comprehension [4, 5]. It introduces three pre-MODIFY activities:

- **PREDICT** ∈ **ANALYZE**: Analyze code/artifact to anticipate how it will behave; formulate a **hypothesis** about *expected behavior* (aka, *expected outcomes*);
- **RUN** ∈ **APPLY**: Run experimental procedure (execute code) to **test the hypothesis**, so as to observe the *actual behavior* of the code/artifact (under given conditions);
- **INVESTIGATE** ∈ **EVALUATE**: Analyze experimental results (*expected vs actual*) to (in)validate the hypothesis; explain outcome, incl. how the code/artifact appears to work.

In concert, these three initial activities form a support framework: they reduce cognitive load, focus attention on essential features, and establish a shared foundation. Only once this scaffolding is in place do learners move on to the more creative phases of **PRIMM**: **MODIFY** and **MAKE** (where the latter is equivalent to **CREATE**).

The Scientific Method. The three initial pre-MODIFY stages parallel the steps of the *scientific method*, specifically its core cycle of hypothesis formulation, empirical testing, and result interpretation:

- **HYPOTHESIZE**: Propose an explanation or prediction;
- **TEST**: Design and run an experiment or study; and
- **EVALUATE**: Interpret results: *accept/reject/refine* hypothesis.

The, almost playful, “*what happens if...*” hypothesis-testing is essentially **Exploratory Inquiry-Based Learning (IBL)** which is *learning through the iterative cycle of hypothesis generation and testing, modeled after the scientific method*:

- **INQUIRE**: Pose questions to generate hypotheses;
- **EXPLORE**: Test them systematically; and
- **LEARN**: Refine understanding based on evidence.

Learners can subject AI-generated code through this process, probing it from various perspectives to *learn* about the code and, crucially, *its properties*. This works for both *functional* properties (*behavior, interaction, authentication, error handling*) and *non-functional* properties (*performance, security, privacy, safety, modularity, scalability, assessability, biases, fairness*) to gain overall *trustworthiness* of otherwise inherently *untrustworthy* generated AI-code. In doing so, our students move up the cognitive process ladder.

AI Competences. AI-empowered human programmers (student & professional) will need to learn *AI competences* in order to:

- **PROMPT** ∈ **SYNTHESIS**: Formulate clear and unambiguous input specifications to guide AI to generate intended code;
- **REVIEW** ∈ **EVALUATE**: Critically assess AI-generated code to ensure it aligns with the intended purpose.

They will need to learn how *effectively* **PROMPT**; *input* to the AI. More importantly, however, they will need to learn how to *critically* **REVIEW** the *output* from the AI. This is precisely where the initial **PRIMM** phases offer us with a concrete recipe; an opportunity for *exploratory inquiry-based learning* (beyond mere **USE**):

REVIEW = **PREDICT** → **RUN** → **INVESTIGATE** ∈ **ANALYZE** → **APPLY** → **Eval**; once we are satisfied with the AI-code through critical **REVIEW**, it must still undergo rigorous V&V (Verification & Validation), as with human-written code.

In summary, generative AI turns UMC “upside down”, but we can design for this shift so that learning moves *up* the competence ladder toward the scientific method and inquiry-based learning.

Author’s accepted manuscript version of ICSE-SEET 2026 2-page extended abstract.

REFERENCES

- [1] Lorin W. Anderson and David R. Krathwohl. 2001. *A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom’s Taxonomy of Educational Objectives*. Longman, New York.
- [2] Benjamin S. Bloom, Max D. Engelhart, Edward J. Furst, Walker H. Hill, and David R. Krathwohl. 1956. *Taxonomy of Educational Objectives: The Classification of Educational Goals. Handbook I: Cognitive Domain*. Longmans, Green and Co, New York.
- [3] Irene Lee, Fred Martin, Jill Denner, Bob Coulter, Walter Allan, Jeri Erickson, Joyce Malyn-Smith, and Linda Werner. 2011. Computational Thinking for Youth in Practice. *ACM Inroads* 2, 1 (Feb. 2011), 32–37. <https://doi.org/10.1145/1929887.1929902>
- [4] Fred Martin, Irene Lee, Nicholas Lytle, Sue Sentence, and Natalie Lao. 2020. Extending and evaluating the use-modify-create progression for engaging youth in computational thinking. In *Proceedings of the 51st acm technical symposium on computer science education*. 807–808.
- [5] Sue Sentence and Jane Waite. 2017. PRIMM: Exploring pedagogical approaches for teaching text-based programming in school. In *Proceedings of the 12th Workshop on Primary and Secondary Computing Education*. 113–114.
- [6] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention Is All You Need. In *Advances in Neural Information Processing Systems 30 (NIPS 2017)*. 5998–6008.